

When Training Data Are Gone: Selective Integration of Pretrained Intrusion Detectors

Abstract—Effective intrusion detection benefits from diverse attack data, yet raw historical source records may no longer be accessible due to privacy and retention constraints, preventing further collaborative training. In this paper, we study selective integration of authorized pretrained detectors when raw source records are unavailable but coarse source class counts remain available for reuse. Under a hard deployment budget, the goal is to choose a fixed detector set that preserves diverse source class coverage while maintaining predictive quality in the deployment domain. We present SILOSTITCH, which gives weighted source class coverage strict priority over deployment-domain predictive behavior rather than combining them into a single score. SILOSTITCH aligns heterogeneous detector outputs and uses held-out deployment-domain calibration records to assess deployment-domain predictive behavior. It then exactly maximizes weighted source class coverage under a logical byte budget via dynamic programming, refines the selected set without reducing coverage, and fits a shared stacker. To limit privacy leakage, we also protect uploaded score blocks via local differential privacy. Theoretically, we prove that the calibration bounds remain valid after adaptive selection, that the primary dynamic program is exact, and that perturbed scores satisfy label-conditional local differential privacy. Across four intrusion detection datasets with more than 60 candidate detectors per task, SILOSTITCH reduces modeled logical communication by approximately half under a 50% selection-cost budget while preserving maximum weighted source class coverage and near-full-pool detection quality.

I. INTRODUCTION

Effective intrusion detection relies on training data that capture diverse traffic conditions and attack behaviors. However, data collected by a single organization typically reflect only its own operational environment, limiting the diversity of attack patterns available for training. Collaborative learning, particularly Federated Learning (FL), can mitigate such data silos by learning from distributed records without pooling them centrally [1]–[4]. Yet these approaches still require contributors to retain local training records and participate in additional model optimization. In practice, raw historical network logs may become inaccessible because they can contain sensitive operational and personally identifiable information [5], [6], and may need to be deleted under privacy and retention requirements such as the GDPR [7].

When these raw records are unavailable but authorized pretrained detectors and coarse source class counts remain,

reusing the detectors offers a way to exploit complementary source knowledge without revisiting the original data [8]. However, existing approaches either select or aggregate pretrained models without explicitly accounting for heterogeneous deployment costs [9], [10], or perform cost-aware detector selection rather than choosing one fixed detector set for deployment [11]. This gap motivates our research question: *How can a coordinator selectively integrate a fixed set of pretrained intrusion detectors under a deployment budget while preserving source class coverage and maintaining detection performance in the deployment domain?*

Answering this question presents three challenges. First, detectors contributed by different parties may use different label spaces and support different target classes, making their outputs difficult to compare and integrate directly. Second, source class coverage and deployment-domain detection performance may favor different detectors: a detector with unique class coverage may no longer perform well after distribution shifts, whereas a detector that performs well in the deployment domain may add little new coverage. The coordinator should therefore preserve unique source coverage without filling the limited budget with detectors that are weak or redundant in the deployment domain. Third, detector selection must satisfy the deployment budget while remaining statistically reliable and privacy-aware. Because calibration data are used to choose detectors, their performance estimates must remain valid after selection, while uploaded prediction scores may still reveal sensitive information.

To address these challenges, we present SILOSTITCH, a framework for selective integration of pretrained intrusion detectors under deployment budgets. The key idea is to prioritize source class coverage over deployment-domain predictive performance: SILOSTITCH first maximizes weighted source class coverage under the deployment budget and then improves deployment-domain predictive behavior without reducing that coverage. The two objectives use different information: coarse source class counts indicate which target classes were represented during training, whereas held-out deployment-domain calibration records indicate how well the pretrained detectors perform after distribution shift [12]. SILOSTITCH first aligns heterogeneous detector outputs to a common label space and uses the calibration records to assess their deployment-domain predictive behavior. It then solves the primary coverage problem through dynamic programming, refines the selected set without sacrificing coverage, and integrates the selected outputs with a shared stacker [13].

We prove that the calibration bounds remain valid after

adaptive detector selection and that the primary dynamic program is exact. Because avoiding raw-data sharing alone does not eliminate privacy risks from model outputs [14], SILOSTITCH also supports optional Laplace perturbation before score upload. The perturbed scores satisfy record-level, label-conditional local differential privacy; this guarantee applies to the uploaded scores, not to public labels or contributed detector parameters.

We evaluate SILOSTITCH on four intrusion detection datasets with over 60 candidate detectors per task. At a 50% selection-cost budget, SILOSTITCH maximizes weighted source class coverage while reducing modeled logical communication by 49.3% to 50.4%, demonstrating that diverse source coverage can be retained at roughly half the communication cost. Importantly, these savings incur only a small loss in detection quality: across all four datasets, the macro F1 gap from the full detector pool is limited to 0.028. In all 40 enumerable comparison instances, the primary solver attains the same coverage as exhaustive search. In summary, we make four contributions.

- We formulate budget-constrained selective integration of pretrained intrusion detectors when raw historical source records are unavailable and introduce an ordered objective that prioritizes source class coverage over deployment-domain predictive behavior.
- We develop SILOSTITCH to align heterogeneous detectors, exactly maximize weighted source class coverage under the budget, refine the selected set without reducing coverage, and integrate their outputs with a shared stacker.
- We prove that the calibration bounds remain valid after adaptive detector selection, that the primary coverage problem is solved exactly, and that local refinement preserves this optimum; we also establish label-conditional local differential privacy for randomized score uploads.
- We evaluate SILOSTITCH on four intrusion detection tasks, showing that a 50% selection-cost budget roughly halves modeled logical communication while preserving maximum source class coverage and near-full-pool detection quality.

II. BACKGROUND AND RELATED WORK

We first review existing collaborative training and model integration methods, and then introduce the notion of differential privacy.

Retraining and Adaptation of Intrusion Detectors. When participants retain local training data and can rerun optimization, federated learning coordinates their parameter updates without transferring raw data [1]–[3]. Within intrusion detection, federated methods apply collaborative training across distributed network environments [6], [18]. Recent NIDS research also redesigns the training pipeline to withstand adverse data conditions. For example, RAPIER handles low-quality labels for encrypted malicious traffic [19], Rosetta uses TCP-aware augmentation to improve robustness across network environments [20], and CoLD denoises labels before

fitting a classifier [21]. These approaches improve training or adaptation when the underlying records remain available to the learning procedure. In contrast, we consider the setting where the raw historical records are unavailable, while the pretrained detectors and coarse class-count metadata remain authorized for reuse.

Pretrained Model Integration. When source training data are no longer available but their trained models remain accessible, pretrained model integration is the closest line of work. ZooD ranks a heterogeneous model zoo by cross-domain transferability and aggregates its top K models [9], while Model-GLUE studies selection and aggregation strategies for heterogeneous pretrained model zoos [10]. However, these methods focus on transfer or integration utility rather than the hard aggregate selection-cost constraint studied here.

Output-Level Ensembles and Cost-Aware Selection. Once detector outputs are available, stacked generalization can train a downstream classifier over them [13]. SIRAJ aggregates outputs from heterogeneous threat-intelligence sources using a pretrain-and-fine-tune framework [15], while FedMStacking uses stacking to support heterogeneous local models with misaligned labels [17]. To account for inference cost, SPIREL learns a per-item policy that trades off model invocations against classification utility [11]. These approaches cover complementary pieces of our setting: output ensembles combine available predictions, while SPIREL controls cost through per-item invocation decisions. SILOSTITCH instead makes a one-time subset decision after deployment-domain screening under a hard aggregate budget and then fits a shared classifier over the selected outputs.

Table I summarizes the combination of properties most relevant to our deployment setting: deployment-domain screening, fixed subset selection under a hard aggregate budget, source-class priority, and shared stacking over only the selected outputs. This combination motivates the ordered selection problem formalized next.

Local Differential Privacy. Prediction outputs can reveal information about their inputs, so a privacy guarantee must specify both where randomization occurs and which fields it protects. For example, PrivateKT perturbs locally produced predictions before uploading them for knowledge transfer [16], whereas differentially private federated trees protect the statistics exchanged while training a new model [22]. Our setting differs because SILOSTITCH uploads scores from selected detectors to fit a shared classifier while treating labels as public. We therefore specialize the local model of Duchi et al. [23] to this release.

Definition II.1 (Label-conditional local differential privacy). For a fixed public label y , a contributor applies a randomized mechanism $\mathcal{M}_y$ to the feature vector x . The upload $\mathcal{R}(x, y) = (\mathcal{M}_y(x), y)$ satisfies label-conditional $(\epsilon, 0)$ -local differential privacy if, for every fixed y , every pair x, x' , and every measurable output set $\mathcal{O}$,

$$\Pr[\mathcal{M}_y(x) \in \mathcal{O}] \leq e^\epsilon \Pr[\mathcal{M}_y(x') \in \mathcal{O}]. \quad (1)$$

Table I
COMPARISON WITH RELATED METHODS. ✓ DENOTES DIRECT SUPPORT, ● RELATED SUPPORT, AND ○ NO SUPPORT.

Method	Model Integration			Subset Selection				Privacy	
	Pretrained Models	Heterogeneous	Score Stacking	Deployment Screening	Fixed Set	Hard Budget	Class Priority	Score	LDP
SIRAJ [15]	○	✓	●	○	○	○	○	○	○
PrivateKT [16]	○	○	●	○	○	○	○	✓	○
SPIREL [11]	✓	✓	●	○	●	●	○	○	○
ZooD [9]	✓	✓	●	○	✓	●	○	○	○
FedMStacking [17]	○	✓	✓	○	○	○	○	○	○
SILOSTITCH	✓	✓	✓	✓	✓	✓	✓	✓	✓

This definition compares records with the same public label and protects the feature-derived portion of the upload, and the label itself remains public.

To instantiate Definition II.1, let a deterministic score map ψ have replacement sensitivity

$$\Delta_1(\psi) = \sup_{x, x'} \|\psi(x) - \psi(x')\|_1. \quad (2)$$

Adding independent noise $\eta_k \sim \text{Lap}(0, \Delta_1(\psi)/\epsilon)$ to every coordinate yields a mechanism that satisfies Definition II.1 [24]. If one record releases detector score vectors with parameters $\epsilon_1, \dots, \epsilon_s$, sequential composition gives the parameter $\sum_{i=1}^s \epsilon_i$. Under one record replacement, releases derived from unchanged records do not increase this privacy parameter. In SILOSTITCH, this mechanism applies to the selected detector score vectors before they are assembled into uploaded score blocks. Theorem V.1 establishes label-conditional local privacy for the resulting score upload under the public label adjacency relation.

III. PROBLEM STATEMENT

In this section, we formalize the problem of selective integration of pretrained intrusion detectors when the raw historical source records used to train them are unavailable. We first define the information available for detector reuse, map heterogeneous detector outputs to a common deployment label space, and model the communication cost of deploying a selected subset. We then specify the threat model for score uploads and formulate a budget-constrained selection objective that prioritizes source class coverage before deployment-domain predictive behavior.

A. System Model

We consider J contributors that provide a pool of M pretrained intrusion detectors for integration. The raw records originally used to fit these detectors are no longer available to the integration process, while the metadata needed to interpret and deploy the detectors remain available. For candidate i , let f_i denote the pretrained detector, with native probability output $p_i(x) \in \mathbb{R}_{\geq 0}^{h_i}$ satisfying $\mathbf{1}^\top p_i(x) = 1$, serialized model size K_i bytes, and coarse source class-count vector $H_i = (H_{i,1}, \dots, H_{i,C})$ after semantic mapping, where $H_{i,c}$ records the source training count mapped to target class

c . Because contributors may use different class definitions, we also associate each candidate with a semantic map that specifies how its native outputs correspond to the deployment label space.

We first make these heterogeneous outputs comparable. Let the deployment label space contain C target classes. For candidate i , the coordinator uses a semantic map

$$A_i \in \{0, 1\}^{(C+1) \times h_i}, \quad \mathbf{1}_{C+1}^\top A_i = \mathbf{1}_{h_i}^\top, \quad (3)$$

so each native output coordinate maps to exactly one aligned coordinate, while multiple native coordinates may map to the same target class. The coordinator then computes the aligned probability vector

$$q_i(x) = A_i p_i(x) \in \Delta^C, \quad (4)$$

where Δ^C is the probability simplex with $C + 1$ coordinates. The first C coordinates correspond to the predefined target classes. We reserve the final coordinate, denoted by $\perp$, for probability mass that cannot be mapped to any target class. By keeping this unmapped mass explicitly, later evaluation penalizes it instead of silently dropping it. We refer to $q_i(x)$ as candidate i 's aligned score vector for record x . For a record set $X = \{x_t\}_{t=1}^{|X|}$, we stack these vectors as

$$Q_i(X) = [q_i(x_t)]_{t=1}^{|X|} \in \mathbb{R}^{|X| \times (C+1)}, \quad (5)$$

which we call the aligned score block of candidate i on X .

After making the detector outputs comparable, we use held-out deployment records to determine whether each pretrained detector remains useful under the deployment distribution. Before selection, the coordinator therefore holds a labeled calibration set $\mathcal{D}^{\text{cal}}$ from the deployment domain. These records are used only to evaluate the pretrained candidate pool and are disjoint from the records later used to fit the shared stacker and from the sealed test set. Every detector, semantic map, and source class-count vector is fixed before calibration begins. The test labels remain hidden until both detector selection and stacker fitting are complete, so they cannot influence detector selection or stacker fitting.

We next model the selection cost of choosing a detector. A selected detector must be delivered for use, and its aligned score block must later be uploaded for shared stacker fitting. We combine these two communication terms into a single selection cost measured in logical bytes. Here, logical bytes

count application-level payload under the modeled workflow and exclude transport headers and implementation-specific protocol overhead. Let B_t denote the total number of stacker-fitting records across all contributors, and let d denote the logical byte width assigned to one uploaded scalar, including either a probability value or a public label. For a selected set S , we define

$$C_{\text{sel}}(S) = \sum_{i \in S} b_i, \quad b_i = JK_i + dB_t(C + 1). \quad (6)$$

The first term in b_i accounts for delivering candidate i to the J contributors, while the second accounts for uploading the aligned score block over all stacker-fitting records.

Given a budget fraction $\beta \in (0, 1]$, we set the hard selection-cost budget to

$$B = \left\lfloor \beta \sum_{i=1}^M b_i \right\rfloor. \quad (7)$$

The reference cost includes the entire candidate pool, including candidates that may later fail calibration. We additionally count the initial candidate upload and the stacker-label upload:

$$C_{\text{total}}(S) = \sum_{i=1}^M K_i + C_{\text{sel}}(S) + dB_t. \quad (8)$$

We use C_{sel} to enforce the selection-cost budget and C_{total} to compare the overall logical communication of SILOSTITCH with that of the full detector pool. This accounting covers integration and stacker fitting; distribution of the fitted stacker and subsequent inference traffic are outside the modeled communication cost.

B. Threat Model

We consider the privacy risk introduced when contributors upload detector scores for shared stacker fitting. Although raw historical source records are unavailable and never transferred during integration, the prediction scores computed from stacker-fitting records may still reveal information about those records. We therefore consider an honest-but-curious coordinator that follows the prescribed integration protocol but may attempt to infer record-level information from the uploaded scores.

We assume that each contributed detector, its semantic map, and its coarse source class counts are fixed and honestly reported before selection. Record labels used for stacker fitting are treated as public. When score protection is enabled, each contributor randomizes its selected score blocks locally before transmission, so the privacy guarantee does not rely on the coordinator performing the randomization. Our guarantee protects the feature-dependent information contained in these randomized score uploads; it does not protect the public labels, contributed detector parameters, or source class-count metadata. Malicious contributors that poison detector parameters, forge semantic maps or source class counts, or deviate from the prescribed randomization mechanism are outside our scope, as we focus on record-level privacy of score uploads.

C. Detector Selection Objective

The aligned outputs let us compare detectors in a common label space, but the budget may still prevent us from deploying all of them. We therefore need to decide which subset to keep. Two considerations matter: we want the selected set to preserve source classes represented across the source domains, and we also want its detectors to remain useful in the deployment domain. These goals can favor different candidates. For example, one detector may be the only affordable model that covers a rare source class but have only moderate deployment-domain performance, while another may exhibit strong deployment-domain performance but add no new source-class coverage.

We handle this conflict with a strict priority: weighted source class coverage is maximized first under the selection-cost budget, and deployment-domain predictive behavior is compared only after that maximum is fixed. This is a lexicographic objective: the secondary objective can improve the choice among equally well-covered sets, but it can never compensate for losing primary coverage. We express this priority as

$$\text{lexmax}_{\emptyset \neq S \subseteq \hat{\mathcal{R}}} (C_{\text{src}}(S), \Phi(S)) \quad \text{s.t.} \quad \sum_{i \in S} b_i \leq B. \quad (9)$$

Here, $\hat{\mathcal{R}}$ is the pool that passes calibration, C_{src} is the weighted source class coverage defined later, and Φ summarizes deployment-domain predictive behavior. We solve the maximum source-coverage problem with a bitmask dynamic program and then apply coverage-preserving local refinement to improve deployment-domain predictive behavior. Section V characterizes this refinement and explains why a single fixed weighted sum cannot enforce the same priority across arbitrary candidate pools.

IV. DESIGN OF SILOSTITCH

A. Overview of SILOSTITCH

We now describe how SILOSTITCH realizes the preceding problem formulation. Figure 1 summarizes the workflow in four steps. In Step 1, we use the semantic map A_i defined in the system model to transform each detector's native probabilities $p_i(x)$ into the aligned representation $q_i(x)$, so that heterogeneous detector outputs can be compared in a common label space. We also retain the source class-count metadata and selection cost introduced earlier for subsequent selection. These source counts are later converted into the source-support information used by the primary objective.

With the detector outputs made comparable, Step 2 uses held-out calibration records to evaluate how the pretrained detectors perform in the deployment domain. We use an upper confidence bound on Brier risk to determine whether each detector enters the calibration-eligible pool, while retaining additional classwise utility and prediction-pattern statistics for later refinement. Based on this eligible pool, Step 3 first uses the source-support information and detector selection costs to find the maximum weighted source class coverage allowed by

the selection-cost budget. We then use the retained calibration statistics to improve deployment-domain predictive behavior without reducing the achieved coverage. Finally, once the selected detector set is fixed, Step 4 uses only its aligned score blocks to fit one shared stacker and keeps the same selected layout for subsequent sealed-test inference. The following subsections define the operations in these four steps in detail.

B. Selection and Stacking Workflow

Algorithm 1 summarizes the four steps above. Let j index contributors and i index candidate detectors, and write $\mathcal{D}_j^{\text{stk}} = (X_j^{\text{stk}}, Y_j^{\text{stk}})$ for contributor j 's stacker-fitting records and labels. The calibration partition is first used to evaluate the pretrained candidates, form the eligible pool, and compute the deployment-domain statistics used for refinement. The primary solver then selects from this pool using the fixed source meta-data and selection costs. We then refine one primary-optimal solution using the retained calibration statistics. Only after the selected set $\hat{S}$ is fixed do we use the independent stacker-fitting records to train the shared stacker. This separation prevents the records used to fit the stacker from also influencing which detector outputs become its input features.

The contributors then evaluate only the selected detectors on their stacker-fitting records. If score protection is enabled, they randomize the selected aligned score blocks locally before upload; otherwise the same workflow is used without added noise.

C. Brier-Based Evaluation and Screening

After aligning every candidate to the same target label space through $q_i(x)$, we evaluate the aligned outputs on held-out deployment records to obtain comparable statistics for screening and refinement. For this assessment, we use a full-vector Brier score because it is bounded, evaluates the complete aligned probability vector, and naturally penalizes probability mass assigned to the unmapped $\perp$ coordinate.

For calibration record (x_t, y_t) , let $e_{y_t} \in \mathbb{R}^{C+1}$ be the one-hot label vector with a zero $\perp$ coordinate. For candidate i , we define

$$\ell_{i,t} = \frac{1}{2} \|q_i(x_t) - e_{y_t}\|_2^2, \quad g_{i,t} = 1 - \ell_{i,t} \in [0, 1]. \quad (10)$$

Thus, $g_{i,t}$ measures the quality of the whole aligned prediction rather than one coordinate at a time. If a detector places probability on an unmapped output, that probability remains in the vector and lowers the utility.

Eligibility screening. We first determine whether each candidate detector has sufficiently low deployment-domain risk to remain eligible. Using only its empirical mean can be misleading when the calibration set is finite, so we screen with an upper confidence bound on deployment-domain Brier risk. With N calibration records and failure probability $\delta \in (0, 1)$, we compute

$$\hat{\mu}_i = \frac{1}{N} \sum_{t=1}^N g_{i,t}, \quad r = \sqrt{\frac{\log(4M/\delta)}{2N}}, \quad (11)$$

and define

$$R_i^+ = 1 - \hat{\mu}_i + r. \quad (12)$$

We keep candidate i only if $R_i^+ \leq \tau_L$, giving the calibration-eligible pool

$$\hat{\mathcal{R}} = \{i : R_i^+ \leq \tau_L\}. \quad (13)$$

In other words, eligibility depends on both the observed deployment-domain performance and the uncertainty caused by finite calibration data.

Classwise scores. An acceptable average risk does not guarantee that a detector remains useful for every target class. We therefore keep a conservative classwise utility estimate for each detector–class pair and use these scores later when we refine sets that already achieve maximum source class coverage. For class c , let

$$N_c = \sum_t \mathbf{1}\{y_t = c\}, \quad (14)$$

and define

$$\hat{\mu}_{i,c} = \frac{1}{N_c} \sum_{t:y_t=c} g_{i,t}. \quad (15)$$

We require every calibration class to appear at least once, so $N_c > 0$. We then compute

$$r_c = \sqrt{\frac{\log(4MC/\delta)}{2N_c}}, \quad a_{i,c} = [\hat{\mu}_{i,c} - r_c]_+, \quad (16)$$

where $[z]_+ = \max\{0, z\}$. The value $a_{i,c}$ is a lower confidence bound on the deployment-domain utility of candidate i on class c .

Prediction similarity. Two detectors can have similar average utility and still make nearly identical predictions. Selecting both may then consume budget without adding much new predictive information. To capture this redundancy, we compare the aligned outputs of the candidates on a fixed class-stratified set of calibration indices $\mathcal{A} \subseteq \{1, \dots, N\}$. We form

$$z_i = \text{vec}([q_i(x_t)]_{t \in \mathcal{A}}), \quad \kappa_{i,k} = \left[\frac{\langle z_i, z_k \rangle}{\|z_i\|_2 \|z_k\|_2} \right]_0^1. \quad (17)$$

Here, $[z]_0^1 = \min\{1, \max\{0, z\}\}$ and the clipping only removes numerical roundoff outside $[0, 1]$. Later, we use these similarities in a representativeness term: rather than rewarding pairwise difference for its own sake, we ask whether the selected set contains detectors whose prediction patterns represent the wider eligible pool.

D. Coverage-Prioritized Detector Selection

The calibration statistics tell us how the pretrained detectors perform in the deployment domain, but they do not tell us which historical source classes would disappear if particular detectors were removed. That information comes from the coarse source class counts retained with the candidate models. We therefore use source class coverage as the primary objective and use the calibration statistics only after the best achievable coverage has been fixed.

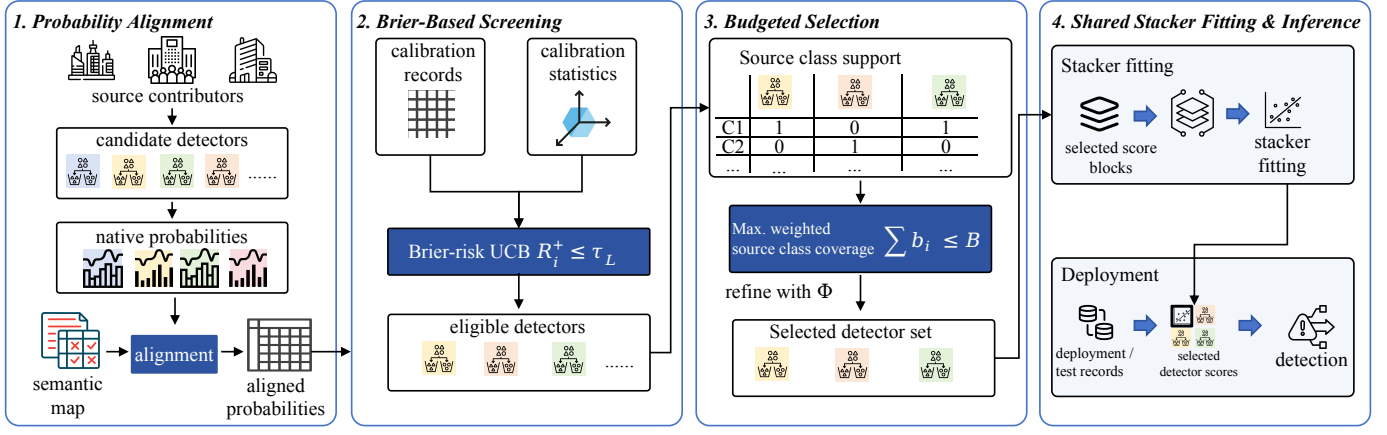


Figure 1. Overview of SILOSTITCH. Step 1 maps the heterogeneous outputs of the pretrained detector pool to a common $C + 1$ -coordinate label space and retains the metadata needed for selection. Step 2 evaluates the aligned detectors on held-out calibration records and forms the calibration-eligible pool. Step 3 exactly maximizes weighted source class coverage under the selection-cost budget and then refines deployment-domain predictive behavior without reducing the achieved coverage. Step 4 fits a shared stacker from the selected aligned outputs and reuses the same selected layout for sealed-test inference.

Algorithm 1: SILOSTITCH Procedure

Require: $\{f_i, A_i, H_i, K_i\}_{i=1}^M$, $\mathcal{D}^{\text{cal}}$, $\{\mathcal{D}_j^{\text{stk}}\}_{j=1}^J$, B , $\epsilon_b \in (0, \infty]$
Ensure: $\hat{S}$, $\hat{f}_{\hat{S}}$

- 1: Construct (Γ, w, b) from the fixed source metadata and cost model
- 2: Evaluate candidates on $\mathcal{D}^{\text{cal}}$ to obtain $\hat{\mathcal{R}}$ and the statistics used by Φ
- 3: Construct Φ from the retained calibration statistics
- 4: $(W^*, S_0) \leftarrow \text{COVER}(\hat{\mathcal{R}}, \Gamma, w, b, B)$
- 5: $\hat{S} \leftarrow \text{REFINE}(S_0, W^*, \Phi, B)$
- 6: Coordinator $\rightarrow$ Contributors: $\{f_i\}_{i \in \hat{S}}$
- 7: **for** $j = 1, \dots, J$ **in parallel do**
- 8: $Z_j \leftarrow \text{concat}_{i \in \hat{S}} Q_i(X_j^{\text{stk}})$
- 9: **if** $\epsilon_b < \infty$ **then**
- 10: $(\Xi_j)_{t,i,k} \stackrel{\text{iid}}{\sim} \text{Lap}(0, 2/\epsilon_b)$
- 11: $\tilde{Z}_j \leftarrow Z_j + \Xi_j$
- 12: **else**
- 13: $\tilde{Z}_j \leftarrow Z_j$
- 14: **end if**
- 15: Contributor $j \rightarrow$ Coordinator: $(\tilde{Z}_j, Y_j^{\text{stk}})$
- 16: **end for**
- 17: $\hat{f}_{\hat{S}} \leftarrow \text{FIT}(\bigcup_j (\tilde{Z}_j, Y_j^{\text{stk}}))$
- 18: **return** $(\hat{S}, \hat{f}_{\hat{S}})$

Weighted source class coverage. Recall that $H_{i,c}$ denotes candidate i 's mapped source count for target class c . A very small count may provide weak evidence that a class was meaningfully represented during training, so we first convert the count into a support indicator. For a minimum support threshold $n_{\min} > 0$, define

$$\Gamma_{i,c} = \mathbf{1}\{H_{i,c} \geq n_{\min}\}. \quad (18)$$

Thus, $\Gamma_{i,c} = 1$ indicates that candidate i has sufficient source

support for class c to count toward the primary objective. Let $\Gamma_i = (\Gamma_{i,1}, \dots, \Gamma_{i,C})$ denote candidate i 's source-support vector.

We also give more weight to source classes that are scarce across the whole candidate pool. Let $H_c = \sum_i H_{i,c}$ denote the aggregate source count of class c , and let $\mathcal{C}_+ = \{c : H_c > 0\}$ denote the source-supported classes. We assume $\mathcal{C}_+ \neq \emptyset$ and set

$$w_c = \begin{cases} \frac{H_c^{-1/2}}{\sum_{c' \in \mathcal{C}_+} H_{c'}^{-1/2}}, & c \in \mathcal{C}_+, \\ 0, & c \notin \mathcal{C}_+, \end{cases} \quad \sum_{c=1}^C w_c = 1. \quad (19)$$

The inverse-square-root rule gives greater weight to less represented source-supported classes while moderating frequency differences relative to inverse-frequency weighting.

For selected set S , we define weighted source class coverage as

$$C_{\text{src}}(S) = \sum_{c=1}^C w_c \mathbf{1}\{\exists i \in S : \Gamma_{i,c} = 1\}. \quad (20)$$

A class contributes once as soon as at least one selected detector provides sufficient source support for it; selecting more detectors for the same class does not increase the primary value. Therefore, C_{src} measures how much historical source class coverage is retained by the selected set, not how accurately that set predicts deployment records.

Secondary predictive objective. We consider the feasible case in which at least one calibration-eligible detector fits the selection-cost budget. After the primary coverage value is fixed, several feasible detector sets may still attain exactly the same coverage. We compare such sets using three secondary objective components that capture different aspects of deployment-domain prediction quality: record-level util-

ity, classwise utility, and prediction-pattern representativeness. With $h_\tau(z) = 1 - e^{-z/\tau}$, define

$$\begin{aligned} U(S) &= \frac{1}{N} \sum_{t=1}^N h_{\tau_U} \left(\sum_{i \in S} g_{i,t} \right), \\ C_{\text{pred}}(S) &= \frac{1}{C} \sum_{c=1}^C h_{\tau_C} \left(\sum_{i \in S} a_{i,c} \right), \\ D_{\text{rep}}(S) &= \frac{1}{|\widehat{\mathcal{R}}|} \sum_{k \in \widehat{\mathcal{R}}} \max_{i \in S} \kappa_{i,k}. \end{aligned} \quad (21)$$

The term $U(S)$ rewards sets whose members perform well across the calibration records. The term $C_{\text{pred}}(S)$ keeps attention on classwise utility by using the conservative scores $a_{i,c}$. The term $D_{\text{rep}}(S)$ rewards sets whose prediction patterns represent those observed across the eligible pool.

The saturating functions in U and C_{pred} impose diminishing returns. Once several selected detectors already perform well on the same record or class, another similar detector contributes less. This makes it harder for redundant prediction patterns to dominate the secondary comparison. We define $\max_{i \in \emptyset} \kappa_{i,k} = 0$ and combine the three terms as

$$\Phi(S) = \lambda_U U(S) + \lambda_C C_{\text{pred}}(S) + \lambda_D D_{\text{rep}}(S), \quad (22)$$

where $\tau_U, \tau_C > 0$, $\lambda_U, \lambda_C, \lambda_D \geq 0$, and $\lambda_U + \lambda_C + \lambda_D = 1$. We use Φ only after the primary coverage value has been fixed, so no gain in deployment-domain predictive behavior can compensate for a loss in source class coverage.

Budgeted selection procedure. We now solve the primary problem: among the calibration-eligible candidates, what is the largest weighted source class coverage that fits within the selection-cost budget? Define

$$\begin{aligned} \mathcal{F}_B &= \left\{ \emptyset \neq S \subseteq \widehat{\mathcal{R}} : \sum_{i \in S} b_i \leq B \right\}, \\ W^* &= \max_{S \in \mathcal{F}_B} C_{\text{src}}(S). \end{aligned} \quad (23)$$

Because C_{src} depends only on which classes are covered, we encode every source-support vector Γ_i as a C -bit mask and run a sparse dynamic program over reachable masks. For each mask, we retain only the least-cost detector set that produces it. Any more expensive set with the same mask is dominated because adding any future detector produces the same updated union mask from either set.

The dynamic program returns both the exact optimum W^* and a least-cost set S_0 that attains it. Starting from this least-cost exact solution leaves as much unused budget as possible for improving deployment-domain predictive behavior. With a tolerance $\varepsilon_{\text{loc}} \geq 0$, we first consider additional candidates according to their secondary gain per byte and accept an addition only when its gain exceeds ε_{loc} . We then examine one-for-one replacements and accept a swap only when it remains budget-feasible, keeps coverage equal to W^* , and improves Φ by more than the same tolerance. Every accepted refinement therefore preserves $C_{\text{src}} = W^*$ while improving Φ .

E. Shared Stacker Learning

Once selection fixes $\widehat{S}$, only the aligned outputs of the selected detectors enter the shared stacker. We use the same fixed detector order for stacker fitting and inference. For input x , we define the selected representation

$$\phi_{\widehat{S}}(x) = \text{concat}_{i \in \widehat{S}} q_i(x) \in \mathbb{R}^{|\widehat{S}|(C+1)}. \quad (24)$$

The selected detector set determines which score blocks are used, while this fixed order determines their feature-column positions.

Local score perturbation. Although raw historical source records are unavailable, the scores computed from stacker-fitting records may still reveal information about those records. When score protection is enabled, each contributor therefore randomizes every coordinate of each selected detector score vector before upload:

$$\tilde{q}_i(x_t) = q_i(x_t) + \eta_{i,t}, \quad \eta_{i,t,k} \stackrel{\text{iid}}{\sim} \text{Lap}\left(0, \frac{2}{\epsilon_b}\right). \quad (25)$$

The resulting stacker input is

$$\tilde{\phi}_{\widehat{S}}(x) = \text{concat}_{i \in \widehat{S}} \tilde{q}_i(x). \quad (26)$$

Section V later proves the corresponding label-conditional local privacy guarantee and its composition across selected detector score vectors.

Shared stacker fitting. Let $\mathcal{D}^{\text{stk}} = \biguplus_{j=1}^J \mathcal{D}_j^{\text{stk}}$ denote the collection of contributor-held stacker-fitting records. After selection, contributors compute the selected score features locally, and the coordinator uses the uploaded features and labels to fit one shared classifier:

$$\hat{f}_{\widehat{S}} = \text{Fit}_{\text{global}} \left(\{(\tilde{\phi}_{\widehat{S}}(x_t), y_t) : (x_t, y_t) \in \mathcal{D}^{\text{stk}}\} \right). \quad (27)$$

For the nonprivate configuration, we simply set $\tilde{\phi}_{\widehat{S}} = \phi_{\widehat{S}}$. Because both configurations use only the columns indexed by $\widehat{S}$, unselected detectors cannot influence the learned stacker.

In a distributed realization of the stacker-fitting step, contributors generate the Laplace noise locally before transmission. Our centralized privacy experiment samples the same distribution only to isolate the utility effect of score randomization. For sealed-test inference, we keep the selected detector set and aligned feature layout fixed and reuse the fitted shared stacker without retraining any contributed detector.

V. THEORETICAL ANALYSIS

We state the guarantees that directly support the workflow: label-conditional privacy of score uploads, post-selection validity of the calibration bounds, exactness of the primary solver, and coverage preservation of local refinement. Proofs and the additional mixture-shift analysis appear in Appendix A.

A. Local Privacy of Score Uploads

Recall that the shared stacker is trained from selected detector scores rather than from the historical source records. Because these scores are still record dependent, we ask what information the contributor-side Laplace mechanism protects. We use the label-conditional adjacency relation defined earlier: the record label is public, and two neighboring records (x, y) and (x', y) differ only in their feature vectors.

Before any stacker-fitting record is processed, we fix the selected detector set and all semantic maps. For a batched upload, neighboring batches have the same size, the same row alignment, and the same sequence of public labels, but they differ in one feature vector. Each contributor samples fresh noise independently across records, selected detector score vectors, and output coordinates. Under this execution model, we obtain the following guarantee.

Theorem V.1 (Label-conditional local score privacy). Suppose that the selected detectors and their semantic maps are fixed and that $0 < \epsilon_b < \infty$, and suppose that each contributor uses fresh private randomness in Eq. 25. Then one released detector score vector satisfies $(\epsilon_b, 0)$ -local differential privacy under the preceding label-conditional adjacency relation. If $s = |\hat{S}|$ detector score vectors are released for the same record, their joint release satisfies $(s\epsilon_b, 0)$ -local differential privacy. Under one-record replacement, releasing the entire contributor batch has the same record-level guarantee. More generally, assigning parameter ϵ_i to detector score vector i gives $(\sum_{i \in \hat{S}} \epsilon_i, 0)$ -local differential privacy.

The theorem also clarifies how the privacy budget interacts with detector selection. If we want one total per-record score budget ϵ , we can assign $\epsilon_b = \epsilon/s$ to each of the s selected detector score vectors. The corresponding noise scale becomes $2s/\epsilon$. Thus, under a fixed total score-privacy budget, selecting fewer detector score vectors also reduces the noise required for each released vector. The guarantee applies only to the randomized score upload, and privacy loss composes again if the same record is released in repeated rounds.

B. Calibration under Heterogeneous Contributors

The privacy result concerns the stacker-fitting partition. Detector selection, in contrast, depends on the separate calibration partition introduced in the system model. We therefore ask a different question here: after we use the calibration statistics to choose a detector subset, can we still trust the confidence bounds computed before selection?

The calibration records may come from different contributors and classes, so we do not assume that every record follows one identical distribution. Instead, let j_t identify the contributor associated with calibration record t , and condition on the observed contributor-label design

$$\mathcal{I} = \{(j_t, y_t)\}_{t=1}^N \quad (28)$$

and on the pretrained candidate pool $\mathcal{Q}$. Let X_t denote the random feature vector for calibration record t , and define

$$G_{i,t} = 1 - \frac{1}{2} \|q_i(X_t) - e_{y_t}\|_2^2 \in [0, 1], \quad (29)$$

whose observed realization is $g_{i,t}$ in Eq. 10. Conditioned on $(\mathcal{Q}, \mathcal{I})$, we assume the records are independent, while allowing their conditional distributions to differ across records. We define

$$\mu_i = \frac{1}{N} \sum_{t=1}^N \mathbb{E}[G_{i,t} \mid \mathcal{Q}, \mathcal{I}], \quad R_i = 1 - \mu_i, \quad (30)$$

and

$$\mu_{i,c} = \frac{1}{N_c} \sum_{t: y_t=c} \mathbb{E}[G_{i,t} \mid \mathcal{Q}, \mathcal{I}]. \quad (31)$$

These quantities average performance over the actual mix of calibration records observed by the coordinator; they are not equal-contributor averages or worst-contributor guarantees.

Theorem V.2 (Simultaneous calibration). Under the preceding assumptions, with probability at least $1 - \delta$, the following inequalities hold simultaneously for every candidate i and class c :

$$R_i \leq R_i^+ \leq R_i + 2r, \quad (32)$$

$$\max\{0, \mu_{i,c} - 2r_c\} \leq a_{i,c} \leq \mu_{i,c}. \quad (33)$$

Consequently, these inequalities remain valid for every candidate in any set selected adaptively from the calibration statistics.

The same event also bounds the effect of calibration screening on feasible source coverage. Appendix A-C gives the corresponding coverage sandwich.

C. Exact Primary Coverage under a Hard Budget

Once calibration fixes the eligible pool, the primary selection problem has a simple interpretation. Each detector has a selection cost and covers a subset of source classes, while the hard budget may prevent us from selecting all detectors at once. We first characterize the difficulty of this problem and then show that the bitmask dynamic program used by SILOSTITCH returns the exact primary optimum.

Theorem V.3 (Primary hardness and exact parameterized solver). Computing W^* is NP-hard, even when every candidate is eligible, every cost is one, and all class weights are uniform. Nevertheless, the bitmask dynamic program returns a nonempty feasible set S_0 such that

$$C_{\text{src}}(S_0) = W^*, \quad (34)$$

with minimum total selection cost among the sets attaining W^* . It stores at most 2^C masks and examines at most $|\hat{\mathcal{R}}|2^C$ state transitions. Thus, the primary stage is fixed-parameter tractable in the number of target classes C . In other words, the transition bound is linear in the number of eligible detectors but exponential in the number of target classes.

D. Secondary Objective and Local Refinement

The exact primary solver fixes the best source coverage that the budget can support, but many detector sets may still attain that same value. We use the secondary objective Φ to choose among them. Before stating the local-refinement guarantee, we first explain why we do not replace the ordered objective with one weighted sum.

A fixed weighted sum cannot enforce the same priority uniformly across deployments, while the secondary objective is normalized, monotone, and submodular for fixed calibration statistics. Appendix A-E provides formal statements and proofs.

This submodularity does not by itself give a global approximation guarantee for the deployed secondary refinement. The family of sets that preserve exact primary coverage,

$$\mathcal{F}^* = \{S \in \mathcal{F}_B : C_{\text{src}}(S) = W^*\}, \quad (35)$$

does not have the simple structure required by standard submodular-knapsack results: removing a detector can destroy the required coverage, and an arbitrary exchange need not restore it. Therefore, familiar global guarantees such as $1 - 1/e$ do not directly apply after we require exact primary coverage.

For $S \in \mathcal{F}^*$, let $\mathcal{N}_1(S)$ contain every set obtained by removing one selected candidate and adding one unselected candidate while preserving budget feasibility and coverage W^* . We can then state a local certificate for a refinement that reaches a one-swap fixed point.

Theorem V.4 (Coverage preservation and one-swap certificate). Let the refinement start from a primary-optimal set $S_0 \in \mathcal{F}^*$ and accept only the budget-feasible additions and coverage-preserving replacements described above. Every set produced by the refinement remains in $\mathcal{F}^*$. Moreover, if the replacement phase ends with a complete pass over all budget-feasible, coverage-preserving one-for-one replacements during which no swap improves Φ by more than ε_{loc} , then its output $\hat{S}$ satisfies

$$\Phi(S') \leq \Phi(\hat{S}) + \varepsilon_{\text{loc}}, \quad \forall S' \in \mathcal{N}_1(\hat{S}). \quad (36)$$

The proof is given in Appendix A-F.

The refinement examines additions and one-for-one replacements rather than all possible multi-detector exchanges. It preserves the primary optimum throughout. If the replacement phase reaches the termination condition in Theorem V.4, the returned set additionally satisfies the stated one-swap certificate.

Historical source coverage does not by itself imply deployment-domain classwise utility. Appendix A-G gives a post-selection analysis under stable contributor–class mean Brier risk and bounded contributor-mixture shift.

VI. EVAL

the shared stacker. The default setting uses a 50% selection-cost budget, $\tau_L = 1$, $n_{\min} = 2$, and equal weights for the secondary components in Φ .

Baselines and reference point. We compare SILOSTITCH with two budget-matched selection baselines and the full detector pool. The first baseline performs random budget-feasible selection by scanning 20 fixed random orders within each split. The second greedily selects detectors according to calibrated Brier utility per selection-cost byte, favoring deployment-domain utility without explicit source-coverage preservation. The full 66-detector pool serves as the unconstrained 100%-cost reference.

Metrics and statistics. We use macro F1 as the primary detection metric because it gives equal importance to all classes under class imbalance. We additionally report accuracy, balanced accuracy, and minimum evaluated class recall, defined over test classes covered by source training. Weighted source class coverage is reported separately to measure retained historical support rather than predictive quality on deployment records. For the DDoS privacy experiment, we report attack rate among benign predictions, i.e., the fraction of benign predictions that correspond to attacks. Communication is computed using Eqs. 6–8. Unless stated otherwise, results are reported as mean values with 95% Student t confidence intervals over five splits, with paired comparisons and Holm correction applied within each evaluation family.

B. RQ1: Detection Quality under a Budget

We first ask whether the 50% selection-cost constraint provides a practically useful deployment point. A useful operating point should attain the primary source-coverage optimum under the budget, reduce communication substantially, and keep the final shared detector close to the full-pool reference. We therefore compare the methods at the 50% operating point and then inspect the full budget sweep in Fig. 2.

Detection quality and source coverage. At the 50% operating point, SILOSTITCH consumes 49.5% to 50.0% of the full-pool selection cost. Across all four tasks, its absolute mean macro-F1 gap from the full detector pool remains below 0.028. The gaps are at most 0.0052 on MalDroid, Darknet, and DoH, while DDoS has the largest gap at about 0.026. Thus, even the largest observed loss remains small relative to the roughly one-half reduction in selection cost.

The budget sweep in Fig. 2 shows that a larger selection-cost budget does not guarantee higher sealed-test macro F1 on every dataset. MalDroid improves most clearly as the budget increases, DoH changes only slightly, and DDoS and Darknet are non-monotonic.

We next examine whether the communication savings degrade performance on the weakest class. Relative to the full pool, the 50% selected set changes accuracy by at most 0.0046 and minimum evaluated class recall by at most 0.0061 on MalDroid, Darknet, and DoH. On DDoS, the accuracy gap is 0.0296 but the minimum evaluated class-recall gap is only 0.0094. Darknet remains weak even under the full-pool reference, which reaches 0.0077 minimum evaluated class

recall compared with 0.0058 after selection. Most importantly for our primary objective, SILOSTITCH attains the primary optimum W^* under the 50% budget on all four datasets.

Table III

MACRO F1 AT THE 50% SELECTION-COST BUDGET. RANDOM SELECTION AND INDIVIDUAL-UTILITY RANKING RECEIVE THE SAME BUDGET; THE FULL DETECTOR POOL IS AN UNCONSTRAINED REFERENCE. ENTRIES REPORT MEANS AND 95% STUDENT t CONFIDENCE INTERVALS OVER FIVE REPEATED SPLITS.

Method	DDoS	MalDroid	Darknet	DoH
Full detector pool	0.6412 ± 0.0121	0.8449 ± 0.0123	0.5972 ± 0.0128	0.6073 ± 0.0042
Random selection	0.6079 ± 0.0287	0.8344 ± 0.0100	0.5931 ± 0.0071	0.6074 ± 0.0028
Individual utility ranking	0.6303 ± 0.0437	0.8374 ± 0.0132	0.5921 ± 0.0168	0.6069 ± 0.0042
SILOSTITCH	0.6153 ± 0.0453	0.8397 ± 0.0121	0.5953 ± 0.0148	0.6064 ± 0.0038

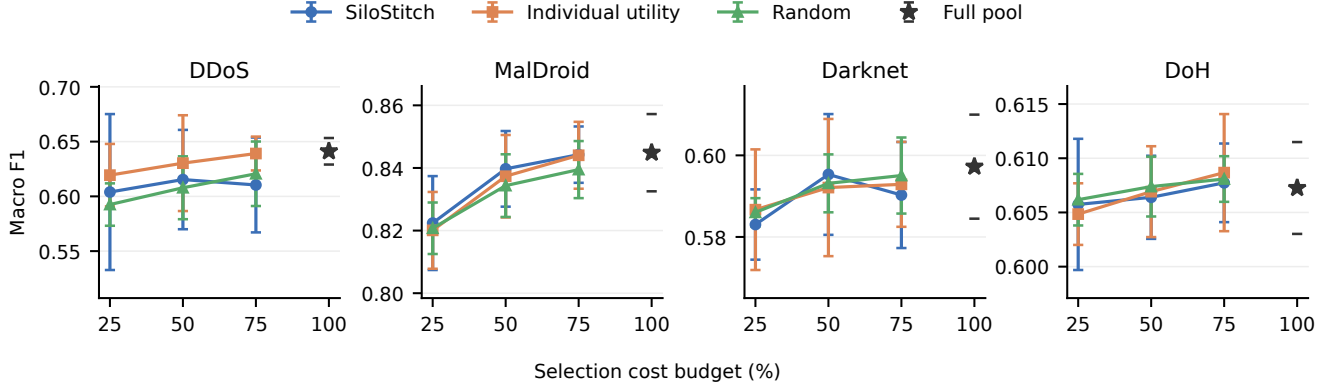


Figure 2. Macro F1 as a function of the selection-cost budget. We compare SILOSTITCH with the two budget-matched selection rules at 25%, 50%, and 75%; the star marks the unconstrained pool of 66 detectors. Error bars are 95% confidence intervals over five repeated splits after averaging the 20 random selections within each split. Each dataset uses its own vertical scale.

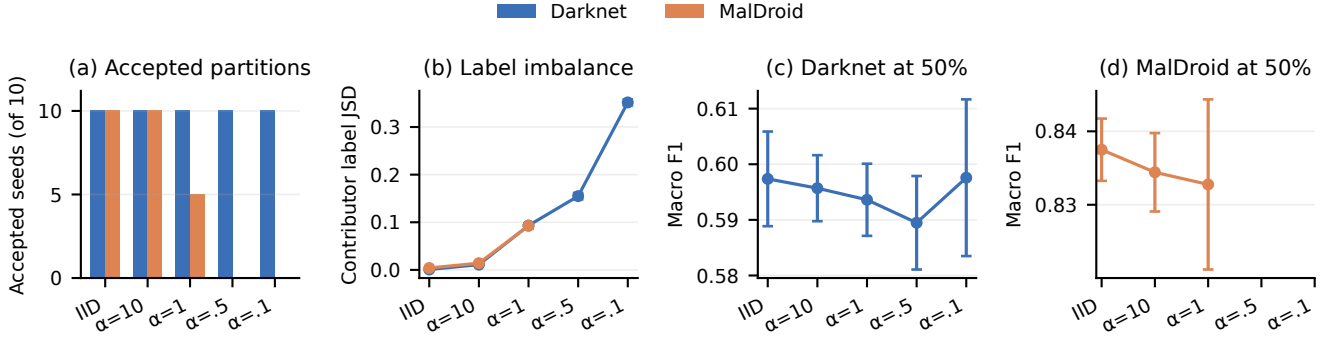


Figure 3. Effect of source-contributor label heterogeneity at the 50% selection-cost budget. Smaller Dirichlet α indicates stronger imbalance

Table IV
MACRO F1 AT THE 50% SELECTION-COST BUDGET FOR EACH SOURCE DETECTOR AND STACKER COMBINATION. ENTRIES ARE MEANS OVER FIVE SPLITS; BOLD MARKS THE LARGEST DESCRIPTIVE MEAN FOR EACH DATASET.

Dataset	XGB →XGB	LGBM →XGB	XGB →LGBM	LGBM →LGBM
DDoS	0.617	0.635	0.633	0.615
MalDroid	0.806	0.833	0.827	0.840
Darknet	0.578	0.585	0.592	0.595
DoH	0.592	0.591	0.607	0.606

the maximum available source coverage, the hard priority is non-binding; when a utility-oriented selector would trade coverage away, the priority changes the result and prevents that loss.

D. RQ3: Selection Components and Learners

After the primary coverage value has been fixed, SILOSTITCH still needs to choose among coverage-equivalent detector sets and then fit the shared stacker. RQ3 therefore separates design choices at these two levels. We first remove the three secondary objective components one at a time, then vary the source-coverage settings, and finally compare the source detector and stacker backends.

Secondary objective components. Recall that U measures record-level utility, C_{pred} measures conservative classwise utility, and D_{rep} measures how well the selected set represents the prediction patterns of the eligible pool. In every ablation, we keep the primary source class coverage fixed at its optimum and remove only one of these secondary terms. At the 50% budget, removing U lowers DDoS macro F1 by 0.0190, while removing D_{rep} raises it by 0.0181. Across datasets and budgets, however, the direction of these changes is not consistent, and no single removal remains statistically significant after Holm correction. We therefore interpret these terms as tie-breakers among coverage-equivalent sets rather than as individually universal sources of accuracy gain.

Source-coverage settings. We next vary the minimum source count $n_{\text{min}} \in \{1, 2, 5, 10\}$ and replace the inverse-square-root class weights with uniform weights. Across two fixed Darknet layouts and two learner combinations, these changes leave the selected set, attained source coverage, macro F1, and minimum evaluated class recall unchanged. The tested Darknet result is therefore insensitive to these values of n_{min} and to the two evaluated class-weighting rules.

Learning backends. Table IV shows that backend choice creates larger descriptive differences than the source-coverage settings above

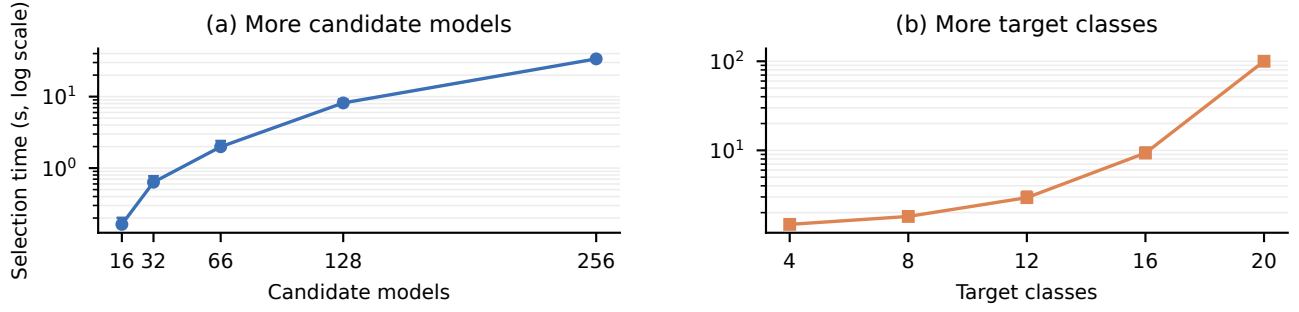


Figure 4. Scaling of the primary selector. Panel (a) fixes the class count at 12 and increases the candidate pool, while panel (b) fixes 66 candidates and increases the class count. The vertical axis is logarithmic; points are medians and upper bars are 95th percentiles over ten repetitions.

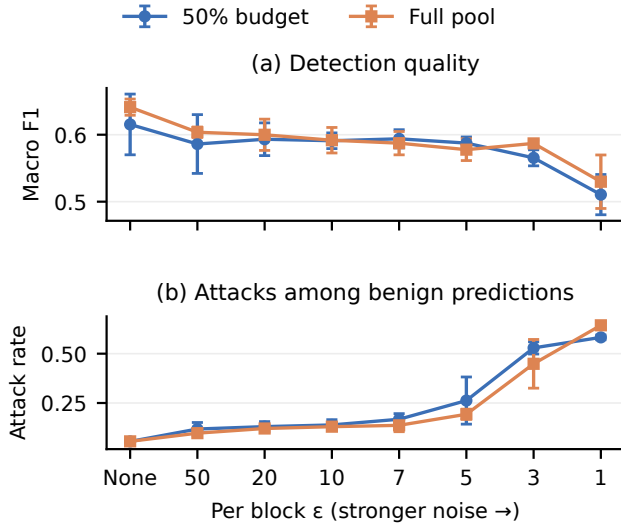


Figure 5. Effect of Laplace score randomization on DDoS. Panel (a) reports macro F1 and panel (b) reports the attack rate among benign predictions, i.e., the fraction of predictions labeled benign that are attacks. From left to right, the per-block privacy parameter decreases and noise becomes stronger. We compare SILOSTITCH at the 50% selection-cost budget with the full detector pool at the same per-block ϵ_b ; their composed per-record privacy parameters differ with the number of released blocks. Error bars are 95% confidence intervals over the same five splits.

ground truth; the larger sweeps below characterize runtime at greater scale.

Scaling behavior. We next vary the two problem dimensions separately in Fig. 4. With the class count fixed at 12, increasing the candidate pool from 66 to 256 raises the median selection time from 1.99 to 33.73 s. With 66 candidates fixed, increasing the class count from 12 to 20 raises the median from 2.95 to 100.1

REFERENCES

- [1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 54. PMLR, 2017, pp. 1273–1282. [Online]. Available: <https://proceedings.mlr.press/v54/mcmahan17a.html>
- [2] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proceedings of Machine Learning and Systems*, I. Dhillon, D. Papailiopoulos, and V. Sze, Eds., vol. 2. MLSys, 2020, pp. 429–450. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2020/hash/1f5fe83998a09396ebe6477d9475ba0c-Abstract.html
- [3] T. Li, S. Hu, A. Beirami, and V. Smith, "Ditto: Fair and robust federated learning through personalization," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139. PMLR, 2021, pp. 6357–6368. [Online]. Available: <https://proceedings.mlr.press/v139/li21h.html>
- [4] H. Zhao, W. Du, F. Li, P. Li, and G. Liu, "Fedprompt: Communication-efficient and privacy-preserving prompt tuning in federated learning," in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.
- [5] C. S. Johnson, M. L. Badger, D. A. Waltermire, J. Snyder, and C. Skorupka, "Guide to cyber threat information sharing," National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-150, 2016. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-150>
- [6] J. Xu, C. Li, Y. Zheng, and Z. Li, "ENTENTE: Cross-silo intrusion detection on network log graphs with federated learning," in *Network and Distributed System Security Symposium*, 2026. [Online]. Available: <https://www.ndss-symposium.org/wp-content/uploads/2026-s93-paper.pdf>
- [7] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data," Official Journal of the European Union, L 119, pp. 1–88, 2016. [Online]. Available:

APPENDIX A
ADDITIONAL THEORETICAL ANALYSIS

A. Proof of Local Score Privacy

Proof. Because $q_i(x)$ and $q_i(x')$ are probability vectors, their replacement sensitivity satisfies

$$\|q_i(x) - q_i(x')\|_1 \leq 2. \quad (37)$$

Therefore, adding coordinatewise Laplace noise with scale $2/\epsilon_b$ bounds the density ratio of one released detector score vector by e^{ϵ_b} . Sequential composition over the s selected detector score vectors gives the factor $e^{s\epsilon_b}$.

For a batched upload, neighboring batches differ in only one row. Since different rows use independent randomization, parallel composition across the unchanged rows does not increase the record-level privacy parameter. Finally, appending the unchanged public labels does not change the density ratio. $\square$

B. Proof of Simultaneous Calibration

Proof. Conditioned on $(\mathcal{Q}, \mathcal{I})$, Hoeffding's inequality gives

$$\Pr\{|\hat{\mu}_i - \mu_i| > r \mid \mathcal{Q}, \mathcal{I}\} \leq 2e^{-2Nr^2} = \frac{\delta}{2M}.$$

A union bound over the M candidates therefore uses at most $\delta/2$ failure probability. Likewise,

$$\Pr\{|\hat{\mu}_{i,c} - \mu_{i,c}| > r_c \mid \mathcal{Q}, \mathcal{I}\} \leq 2e^{-2N_c r_c^2} = \frac{\delta}{2MC},$$

so a second union bound over all MC candidate–class pairs uses the remaining $\delta/2$. Thus, with probability at least $1 - \delta$, all overall and classwise inequalities hold at the same time.

On this event, $1 - \hat{\mu}_i + r$ lies between R_i and $R_i + 2r$, while $\hat{\mu}_{i,c} - r_c$ lies between $\mu_{i,c} - 2r_c$ and $\mu_{i,c}$. Applying $[\cdot]_+$ gives the classwise result. The key point is that this one event covers all $M + MC$ quantities before selection occurs. We can therefore choose a data-dependent subset without paying another union bound over the 2^M possible detector sets. $\square$

C. Effect of Calibration Screening on Source Coverage

Therefore, the marginal gain from adding the same candidate cannot increase as the selected set grows. For D_{rep} , eligible detector k contributes $\max_{i \in S} \kappa_{i,k}$, whose marginal gain from adding candidate a is

$$\max \left\{ 0, \kappa_{a,k} - \max_{i \in S} \kappa_{i,k} \right\}.$$

As S grows, the existing maximum can only increase, so this gain can only decrease. Averaging and taking a nonnegative weighted sum preserve normalization, monotonicity, and submodularity. $\square$

F. Proof of the Local Refinement Guarantee

Proof. The seed S_0 belongs to $\mathcal{F}^*$. If $S \in \mathcal{F}^*$ and $S \cup \{i\}$ is budget feasible, monotonicity gives $C_{\text{src}}(S \cup \{i\}) \geq W^*$, while the definition of W^* gives the reverse inequality. Hence every accepted addition keeps coverage W^* . Every accepted replacement is explicitly required to be budget feasible and to keep coverage W^* , so all sets produced by the refinement remain in $\mathcal{F}^*$.

During the complete final pass, the current set remains $\hat{S}$ and every set in $\mathcal{N}_1(\hat{S})$ is examined. Since no such swap improves Φ by more than ε_{loc} , Eq. 36 follows. $\square$

G. Coverage Certificates under Contributor Mixture Shift

The source-coverage objective records which classes were represented in the historical source domains. It does not, by itself, guarantee that a detector supporting a class still performs well on that class after deployment. The held-out calibration records provide calibration evidence, but the contributor mixture observed during deployment may itself differ from the mixture seen during calibration. We therefore connect historical source support with classwise predictive utility and quantify how a change in contributor mixture can weaken that connection.

This analysis does not change which set SILOSTITCH selects. Instead, after selection, we use the already computed classwise calibration bounds to ask which historically covered classes also have conservative calibration evidence of classwise utility. For any threshold vector $\mathbf{u} \in \mathbb{R}^C$, define

$$C_{\text{cert}}(S; \mathbf{u}) = \sum_{c=1}^C w_c \mathbf{1}\{\exists i \in S : \Gamma_{i,c} = 1 \wedge a_{i,c} \geq u_c\}. \quad (41)$$

A class contributes to this value only when at least one selected detector both reports enough historical source support and has a conservative calibration utility above the threshold; if $u_c > 1$, no detector can satisfy that class